

EE175AB Final Design Report

Battery Management and Communication System (BMS)

Kaushik Vada Christian Kim Connor Stewart Nick Poon

March 5, 2026

Contents

1	What the project is all about	3
1.1	Executive Summary	3
1.2	Purpose and Application Context	3
1.3	System Photo and Block-Diagram Artifacts	3
1.4	Delivered Subsystems	4
2	Theoretical basis for the project	4
2.1	Battery Monitoring Theory	4
2.2	Current Measurement Theory	4
2.3	Thermistor Temperature Theory	4
2.4	Fan Control Theory	5
2.5	Isolation and Safety Theory	5
3	Design Specifications	5
3.1	Electrical and Battery Specifications	5
3.2	Control and Communication Specifications	5
3.3	Software/Usability Specifications	6
3.4	Safety and Reliability Specifications	6
4	Implementation Strategy	6
4.1	High-Level Architecture (System Block Design)	6
4.2	Repository-Level Decomposition	6
4.3	Communication Strategy	7
4.4	Risk Mitigation Strategy	7
5	Actual Implementation	7
5.1	Embedded Firmware Implementation	7
5.1.1	MCU Platform and Peripheral Bring-Up	7
5.1.2	Battery Monitor Communication	7
5.1.3	Thermistor Acquisition and Conversion	8
5.1.4	Fan Control and Diagnostics	8
5.2	Desktop Software Implementation	8

5.2.1	Runtime/Launcher Layer	8
5.2.2	Serial Parsing and Telemetry Reconstruction	8
5.2.3	Frontend and Interaction Layer	8
5.2.4	Packaging and Distribution	9
5.3	Engineering Process Implementation	9
6	Validation/Verification of Functionality	9
6.1	Verification Approach	9
6.2	Measured/Reported Verification Items	9
6.3	Pass/Fail Conditions	10
6.4	Current Validation Status	10
7	Future Improvements	10
7.1	Firmware Enhancements	10
7.2	Control and Safety Enhancements	11
7.3	Desktop/UX Enhancements	11
7.4	Verification and Documentation Enhancements	11
7.5	Repository-wide artifact review (files, images, PDFs, schematics)	11

1 What the project is all about

1.1 Executive Summary

This project delivers a complete **Battery Management and Communication System (BMS)** for a **10s1p lithium-ion battery pack**. The system combines:

- high-voltage battery sensing and protection,
- an isolated embedded control and communication layer, and
- a desktop dashboard for real-time monitoring, command/control, and firmware update support.

The architecture is intentionally split between a battery domain and a host-computer domain to improve safety and reduce fault propagation risk. On the embedded side, firmware on an STM32F303 platform acquires cell telemetry, measures thermistors, estimates current through a shunt path, controls cooling fan operation with automatic and manual modes, and streams telemetry over USB CDC. On the host side, a PyQt6 application embeds a web frontend to visualize live data and send commands back to the controller.

1.2 Purpose and Application Context

The design targets battery-powered engineering prototypes where safe operation, observability, and controlled interfacing with a development computer are required. Example use cases include:

- laboratory battery characterization,
- educational power electronics and controls experiments,
- embedded systems validation with pack-level telemetry,
- future integration with AI/ML-based state estimation pipelines.

1.3 System Photo and Block-Diagram Artifacts

The repository includes project media used for design communication and implementation traceability, including:

- `Senior Design Block Diagrams.pdf`,
- schematic PDFs for BMS and USB firmware branches,
- GUI screenshots and enclosure renderings referenced in weekly reports,
- firmware and project details documentation in PDF/L^AT_EX form.

1.4 Delivered Subsystems

1. **Embedded Firmware (USB-Connection-Firmware):** BQ76930 communication, current sensing, thermistor conversion, fan control, USB telemetry.
2. **Embedded Firmware (Electronic Load branch):** related STM32 infrastructure for hardware-in-loop and load-control workflows.
3. **Desktop Dashboard (BMS_Dashboard):** PyQt6 launcher, serial parser, command bridge, frontend rendering, packaging/update workflow.
4. **Project Documentation:** weekly reports, architecture summaries, firmware plans, and project detail reports.

2 Theoretical basis for the project

2.1 Battery Monitoring Theory

A 10-cell series lithium-ion pack requires per-cell voltage visibility because pack-level voltage alone cannot detect cell imbalance. The design therefore centers on cell-resolved telemetry and safety checks (OV/UV style fault logic) and uses a dedicated battery monitor IC (BQ76930 family usage documented in firmware and project reports).

2.2 Current Measurement Theory

Pack current is estimated through a shunt resistor and coulomb-counter readout path. In firmware, conversion follows:

$$I_{pack} = N_{LSB} \times I_{LSB}, \quad I_{LSB} = 0.000\,422 \text{ A/LSB}$$

where I_{LSB} is derived from BQ76930 voltage-per-LSB and a 20 mΩ shunt value.

2.3 Thermistor Temperature Theory

Temperature conversion uses a Beta-model NTC equation with parameters:

- $R_0 = 10 \text{ k}\Omega$ at 25 °C,
- $\beta = 3950$,
- divider supply near 3.3 V.

Firmware computes divider resistance and applies a Steinhart/Beta approximation:

$$\frac{1}{T} = \frac{1}{T_0} + \frac{1}{\beta} \ln \left(\frac{R}{R_0} \right)$$

then reports temperature in °C.

2.4 Fan Control Theory

Thermal management uses PWM fan actuation and tachometer feedback. The firmware supports:

- automatic mode with piecewise temperature-to-duty mapping,
- manual duty override command path from GUI,
- hysteresis to suppress threshold chatter,
- moving-average RPM estimation and timeout-based stall detection.

2.5 Isolation and Safety Theory

The project architecture applies galvanic isolation between battery and PC domains so high-energy transients are less likely to couple into user-facing USB interfaces. Safety is layered:

- IC/firmware protections (OV/UV/current/thermal status),
- comparator and hardware disable paths in schematic plan,
- host-side visibility and command arbitration through structured telemetry.

3 Design Specifications

Design goals were defined as measurable, testable specifications compatible with EE175 expectations.

3.1 Electrical and Battery Specifications

Specification	Target / Requirement
Cell configuration	10s1p lithium-ion (Molicel P42A context in project docs)
Nominal pack voltage	Approximately 36 V
Pack capacity (nominal)	4.2 Ah
Per-cell monitoring	10 cell voltages captured and transmitted
Pack current estimate	Shunt-based measurement via BQ76930 current registers
Thermal channels	10 NTC channels sampled through STM32 ADC path

3.2 Control and Communication Specifications

Specification	Target / Requirement
Host interface	USB-C / USB CDC serial telemetry

Default serial speed	115200 baud (GUI and firmware workflow)
Fan control output	PWM control with auto and manual modes
Fan telemetry	Duty, auto/manual state, and RPM reporting
Data protocol robustness	Multi-format parser support and frame assembly resilience
Desktop runtime	PyQt6 launcher with embedded web frontend

3.3 Software/Usability Specifications

- Auto-detect serial port workflow with exclusion rules for known non-target ports.
- Real-time telemetry display for cell voltages, temperatures, current, and fan metrics.
- Bidirectional command path (GUI to MCU) with fallback queue polling.
- Build/package support for Windows, macOS, and Linux release artifacts.

3.4 Safety and Reliability Specifications

- Over-current and thermal fault observability.
- Sensor conversion fault coding for disconnected/shorted/error states.
- Isolation boundary between battery and host domains.
- Reportable system status flags (e.g., SYS_STAT, load presence, fan mode/status).

4 Implementation Strategy

4.1 High-Level Architecture (System Block Design)

The implementation strategy separated development into parallel tracks:

1. **Hardware + firmware track:** establish MCU interfaces, BMS monitor communication, thermal sensing, and fan control.
2. **Desktop software track:** establish serial ingestion, parsing, UI rendering, and command channel.
3. **Documentation + verification track:** weekly technical reports, firmware architecture notes, and evolving project detail documents.

4.2 Repository-Level Decomposition

- `USB-Connection-Firmware/`: primary embedded BMS telemetry/control firmware.
- `Electronic_Load_Firmware/`: related STM32 firmware branch and drivers.
- `BMS_Dashboard/`: desktop app, backend parser/bridge, frontend scene, packaging and updater.

- **Project Details/ and Weekly Reports/:** design intent, progress, and architecture snapshots.

4.3 Communication Strategy

The strategy intentionally used plain serial framing and tolerant parsing to accelerate bring-up:

- firmware emits structured lines and status tokens,
- Python parser reconstructs coherent frames from mixed-line streams,
- frontend consumes normalized dictionaries/JSON-like objects for display,
- command bridge supports GUI-triggered control actions.

4.4 Risk Mitigation Strategy

- Auto-detect BQ address and CRC mode to reduce deployment fragility.
- Treat malformed/mixed serial logs as expected, not exceptional.
- Maintain fallback mock data and command queue mechanisms to continue UI testing when hardware is unavailable.
- Keep firmware and GUI independent enough for staged integration.

5 Actual Implementation

5.1 Embedded Firmware Implementation

5.1.1 MCU Platform and Peripheral Bring-Up

Implementation was built around STM32CubeIDE-generated scaffolding plus custom logic. The firmware includes initialization and runtime usage of:

- I2C for BQ76930 communication,
- ADC channels for NTC sensing,
- timers for PWM fan control and tach period capture,
- USB device CDC interface for host communications.

5.1.2 Battery Monitor Communication

Key implementation details include:

- auto-detection of BMS address candidates (0x08/0x18),
- optional CRC8 support with I2C transactions,
- register read/write wrappers for SYS_STAT/control/protection/current paths,
- conversion utilities for current and voltage reporting.

5.1.3 Thermistor Acquisition and Conversion

The firmware configures ADC sampling with long sample times for higher-impedance thermistor networks, converts raw ADC readings to voltage, then converts voltage to temperature with explicit fault code branches for:

- hardware/timeout conversion errors,
- open-circuit and short-circuit sensor conditions.

5.1.4 Fan Control and Diagnostics

Fan control implementation includes:

- PWM duty mapping to timer ARR/CCR settings,
- tachometer capture for RPM calculation,
- moving average smoothing and timeout handling,
- command-driven manual override.

5.2 Desktop Software Implementation

5.2.1 Runtime/Launcher Layer

`gui_launcher.py` provides a desktop shell with:

- PyQt6 windowing and embedded WebEngine frontend,
- bridge exposure (`sendCommand`) from JS to Python,
- serial worker thread lifecycle management,
- update flow integration and cross-platform icon/resource handling.

5.2.2 Serial Parsing and Telemetry Reconstruction

`backend/data_stream.py` implements robust parsing of line-oriented telemetry using regex-based extraction and frame reconstruction for:

- indexed cell/temperature tokens,
- pack current and fan fields,
- mixed-format line streams,
- stale frame timeout finalization and last-known state propagation.

5.2.3 Frontend and Interaction Layer

The frontend stack (HTML/CSS/JS loaded by the launcher) renders live battery and thermal state while allowing command dispatch to firmware through the QWebChannel bridge and queue fallback path.

5.2.4 Packaging and Distribution

The BMS dashboard repository includes:

- dependency manifests,
- PyInstaller spec and release docs,
- helper scripts for icons and update tooling,
- release links for Windows/macOS/Linux binaries.

5.3 Engineering Process Implementation

Weekly reports track implementation increments such as fan auto-curve tuning, parser upgrades, command reliability improvements, and host runtime fixes. Project details documents preserve architecture decisions and evolving firmware strategy.

6 Validation/Verification of Functionality

6.1 Verification Approach

Validation was executed using staged testing:

1. module-level firmware checks (ADC conversion, BMS register access, PWM control),
2. parser-level host checks on mixed serial logs,
3. GUI integration checks for display refresh and command dispatch,
4. end-to-end bench tests with hardware connected when available.

6.2 Measured/Reported Verification Items

Test Item	Pass Criteria	Observed Outcome
BQ address detection	Correctly locks to valid address	Implemented with dual-address probe and runtime selection
CRC compatibility	Correct read/write framing with/without CRC	Firmware supports both modes and dynamic handling
NTC conversion path	Stable conversion under intended sample timing	Long ADC sample time selected to improve stability
Fan PWM control	Duty updates reflected in fan behavior	Manual and auto duty control paths implemented
Fan RPM telemetry	RPM values reported and filtered	Tach capture + averaging + timeout logic implemented

Serial parser resilience	Reconstructs frames from mixed lines	Multi-format regex parser and pending-frame logic implemented
GUI command reliability	Commands reach firmware consistently	Direct bridge + polling fallback command queue present

6.3 Pass/Fail Conditions

Pass:

- ten cell voltages and thermistor channels are visible in GUI,
- current and fan telemetry update in real time,
- manual fan command changes duty and reports status,
- no critical parser drops under expected mixed serial logs,
- protection/fault indicators can be surfaced to host software.

Fail:

- repeated I2C telemetry corruption with no fallback,
- unstable or non-physical temperature reporting under normal conditions,
- command path deadlock or unresponsive GUI control loop,
- inability to maintain data stream continuity for system state decisions.

6.4 Current Validation Status

Based on weekly-report milestones, the project moved from architecture and enclosure definition (Oct 2025), to firmware and telemetry bring-up (Jan 2026), to parser and GUI reliability hardening plus fan-control integration (Feb 2026), and then toward live hardware integration/calibration and fault-management expansion.

7 Future Improvements

7.1 Firmware Enhancements

- Add stronger packet framing (length + CRC) for all telemetry records.
- Introduce explicit firmware state machine enums with fault latching/clear policy.
- Add calibration storage for thermistor offsets and current-sense gain correction.
- Expand diagnostics counters (I2C retries, parser frame drops, watchdog resets).

7.2 Control and Safety Enhancements

- Implement refined thermal derating logic rather than threshold-only responses.
- Add SOC/SOH estimation module (coulomb count + OCV fusion) for better state reporting.
- Harden redundant hardware trip pathways and post-fault forensic logging.

7.3 Desktop/UX Enhancements

- Add persistent session logging and exportable CSV/JSON datasets.
- Add trend charts for voltage sag, thermal gradients, and fan response.
- Provide explicit alert panes with acknowledgement workflow and fault timeline.
- Add guided configuration pages for thresholds and protection profiles.

7.4 Verification and Documentation Enhancements

- Build a formal test matrix with traceability from requirements to test evidence.
- Automate parser regression tests with recorded serial fixture files.
- Add reproducible bring-up and factory-reset procedures.
- Expand final documentation with a reproducibility appendix (BOM revisions, wiring maps, and known issues).

7.5 Repository-wide artifact review (files, images, PDFs, schematics)

To satisfy complete repository traceability for the EE175 folder, a full-file manifest was generated and reviewed at:

- `Final Report/ee175_artifact_manifest.tsv`

This manifest enumerates every file found under EE175 - Senior Design with path, extension, byte count, and SHA-256 fingerprint prefix. The current manifest snapshot contains 4506 artifacts (excluding itself), spanning firmware source, generated build outputs, dashboard runtime assets, PDFs, CAD/schematic exports, and weekly-report images.

Audit highlights used to inform this report:

- **PDF/schematic set reviewed:** block diagrams, BMS schematics, e-load schematics, firmware architecture/plan PDFs, CubeMX project report PDF, weekly-report PDFs, and syllabus/template PDFs.
- **Image set reviewed:** dashboard branding/icons, weekly-report figures (BMS GUI, enclosure, battery pack, firmware screenshots), and schematic crop exports in firmware temp folders.

- **Firmware source reviewed:** primary STM32 application and USB CDC stack usage in `USB-Connection-Firmware/Core/Src/main.c` and related CubeMX outputs.
- **Dashboard source reviewed:** launcher/backend/frontend implementation for serial ingestion, command bridging, and 3D UI presentation.

Conclusion

This EE175 project demonstrates a complete cross-domain engineering design: battery pack instrumentation, embedded control, host communication, and operator-facing visualization. The repository evidences iterative progress from concept and architecture to integrated firmware/dashboard behavior, with clear opportunities for production-grade hardening in future revisions.